

```
In [1]: #LIST
```

```
In [7]: l3=[3,100,2,50]
l3.sort()
l3
```

```
Out[7]: [2, 3, 50, 100]
```

```
l3=[3,100,2,50] l3.sort(reverse = True) # Hyperparameter tuning l3
```

```
In [9]: l4 =[2,'a',2.2'True]
l4.sort(l4)
```

```
Cell In[9], line 1
      1 l4 =[2,'a',2.2'True]
          ^
```

SyntaxError: unterminated string literal (detected at line 1)

```
In [11]: l5=['z','b','m','c']
l5.sort()
l5
l5.sort(reverse=True)
l5
```

```
Out[11]: ['z', 'm', 'c', 'b']
```

```
In [12]: l1= [10,2.3,(1+2j),True,'nit',[1,2,3]]
```

```
In [ ]: l1[:]
```

```
In [13]: l1[:]
```

```
Out[13]: [10, 2.3, (1+2j), True, 'nit', [1, 2, 3]]
```

```
In [14]: l1[2]=45
l1
```

```
Out[14]: [10, 2.3, 45, True, 'nit', [1, 2, 3]]
```

```
In [ ]: #Touple
```

```
In [15]: t =()
t
```

```
Out[15]: ()
```

```
In [16]: type(t)
```

```
Out[16]: tuple
```

```
In [17]: t1=(10,20,30)
t1
```

```
Out[17]: (10, 20, 30)
```

```
In [20]: t1.count(10)
```

```
Out[20]: 1
```

```
In [21]: t1.count(100)
```

```
Out[21]: 0
```

```
In [22]: t1.index(10)
```

```
Out[22]: 0
```

```
In [ ]: t1
```

```
In [23]: sbi=(1234, 'AQCPM&&*@M')
sbi
```

```
Out[23]: (1234, 'AQCPM&&*@M')
```

```
In [24]: sbi[0]=456
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[24], line 1
----> 1 sbi[0]=456

TypeError: 'tuple' object does not support item assignment
```

```
In [25]: sbi
```

```
Out[25]: (1234, 'AQCPM&&*@M')
```

```
In [27]: l1
```

```
Out[27]: [10, 2.3, 45, True, 'nit', [1, 2, 3]]
```

```
In [28]: t1
```

```
Out[28]: (10, 20, 30)
```

```
In [29]: t1*3
```

```
Out[29]: (10, 20, 30, 10, 20, 30, 10, 20, 30)
```

```
In [ ]: #List and Tuple is Completed
```

```
In [33]: l=[10,20,30]
l*2
```

```
Out[33]: [10, 20, 30, 10, 20, 30]
```

```
In [34]: #SET
```

```
In [36]: s ={}
s
type(s)
```

Out[36]: dict

```
In [40]: s = set()
s
type(s)
```

Out[40]: set

```
In [41]: s1 = {20,30,90,10}
s1
```

Out[41]: {10, 20, 30, 90}

```
In [42]: type(s1)
```

Out[42]: set

```
In [44]: s1.add(15)
s1
```

Out[44]: {10, 15, 20, 30, 90}

```
In [45]: s1.add(5) # sorting works on similar data types only not on heterogeneous data types
s1
```

Out[45]: {5, 10, 15, 20, 30, 90}

```
In [46]: s2 = {5,5.5,'nit',1+2j,True}
s2.add(100)
s2
```

Out[46]: {(1+2j), 100, 5, 5.5, True, 'nit'}

```
In [47]: s3 = s2.copy()
s3
```

Out[47]: {(1+2j), 100, 5, 5.5, True, 'nit'}

```
In [49]: s1
s1.clear()
s1
```

Out[49]: set()

```
In [50]: s3[0]
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[50], line 1
----> 1 s3[0]

TypeError: 'set' object is not subscriptable
```

```
In [51]: s3[:]
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[51], line 1  
----> 1 s3[:]  
  
TypeError: 'set' object is not subscriptable
```

```
In [52]: s3.pop()
```

```
Out[52]: 'nit'
```

```
In [53]: s3
```

```
Out[53]: {(1+2j), 100, 5, 5.5, True}
```

```
In [54]: s3.pop()  
s3
```

```
Out[54]: {(1+2j), 100, 5, 5.5}
```

```
In [57]: s2.discard(1+2j)  
s2
```

```
Out[57]: {100, 5, 5.5, True, 'nit'}
```

```
In [58]: s2.discard(1000) # discard not give error if element is not present in the set  
s2
```

```
Out[58]: {100, 5, 5.5, True, 'nit'}
```

```
In [60]: s2.remove(1000) # remove give error if element is not present in the set  
s2
```

```
-----  
KeyError                                Traceback (most recent call last)  
Cell In[60], line 1  
----> 1 s2.remove(1000) # remove give error if element is not present in the set  
      2 s2  
  
KeyError: 1000
```

```
In [62]: s2.count() # not a function of set  
s2
```

```
-----  
AttributeError                          Traceback (most recent call last)  
Cell In[62], line 1  
----> 1 s2.count()  
  
AttributeError: 'set' object has no attribute 'count'
```

```
In [64]: s2.add(100)  
s2
```

```
Out[64]: {100, 5, 5.5, True, 'nit'}
```

```
In [67]: s2.add(100)  
s2
```

Out[67]: {100, 5, 5.5, True, 'nit'}

```
In [69]: s2.add(100)
s2
```

Out[69]: {100, 5, 5.5, True, 'nit'}

```
In [ ]: #SET OPERATION
```

```
In [71]: s7={1,2,3,4,5}
s8={4,5,6,7,8}
s9={8,9,10}
```

```
In [72]: s7.union(s8)
```

Out[72]: {1, 2, 3, 4, 5, 6, 7, 8}

```
In [73]: s7.union(s8,s9)
```

Out[73]: {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}

```
In [75]: s7 | s8 | s9 # / symbol used for union function
```

Out[75]: {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}

```
In [76]: s8 | s9
```

Out[76]: {4, 5, 6, 7, 8, 9, 10}

```
In [77]: print(s7)
```

{1, 2, 3, 4, 5}

```
In [78]: print(s8)
```

{4, 5, 6, 7, 8}

```
In [79]: print(s9)
```

{8, 9, 10}

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

In []:

In []:

In []:

In []: